

32 — React SPA Patterns in the Admin UI

Requirement: R25 (Admin UI) **Phase:** N

What it is

The Admin UI for this project is a **Single-Page Application (SPA)** built with React. A traditional website loads a new HTML page from the server every time you click a link. An SPA loads one HTML page once, and then JavaScript dynamically rewrites the content as the user navigates. The result feels like a desktop application — instant tab switches, no page flashes, state preserved between views.

Here are the core concepts used in this codebase:

Component composition

React applications are built from **components** — self-contained pieces of UI, each responsible for one thing. Components can contain other components, forming a tree:

```
App
|-- LoginPage (shown when not authenticated)
|-- Dashboard (shown when authenticated)
    |-- HealthStatus
    |-- SearchTest
    |-- ProviderSettings
    |-- PromptEditor
    |-- EtlProgress
    |-- Analytics
    |-- UserManagement
    |-- TestingGuide
```

Each component is a JavaScript function that returns JSX (HTML-like syntax). The parent decides which child to render based on state. This is the React equivalent of "if-else" for the UI.

State management with useState

State is data that changes over time and affects what the user sees. In React, `useState` is the basic tool for declaring state inside a component:

```
const [tab, setTab] = useState("search");
```

This creates: - `tab` — the current value (starts as `"search"`) - `setTab` — a function to change the value

When you call `setTab("providers")`, React re-renders the component with the new value. Every part of the UI that depends on `tab` updates automatically.

JWT-gated routing

This SPA does not use a traditional URL router (like React Router). Instead, it uses a simple pattern: check if a JWT token exists in `localStorage`. If yes, show the Dashboard. If no, show the Login page. This is "JWT-gated routing" — the authentication token acts as the gate between the public and private sections of the application.

Centralized API module with token injection

All HTTP requests to the backend go through a single `api.js` module. This module: 1. Reads the JWT token from `localStorage` 2. Attaches it as a `Bearer` token in the `Authorization` header 3. Handles 401 responses globally (clears the token and forces a page reload)

This centralization means individual components never handle auth headers or token expiry — they just call `await listConfig()` or `await searchCategories(query)` and the module handles the rest.

Tab-based navigation

Instead of URL-based routing (where `/settings` loads the settings page), this SPA uses tab state. A `tab` variable holds the current section identifier (`"search"`, `"providers"`, `"prompts"`, etc.), and the Dashboard conditionally renders the corresponding component. This is simpler than a full router and sufficient for an admin panel where bookmarkable URLs are not a priority.

Real-life analogy

Think of the Admin UI as a **physical binder with tabbed dividers**.

A traditional multi-page website is like having separate binders for each topic. To switch from "Search" to "Settings," you have to walk to the shelf, put one binder back, and pull out another. Each transition involves a trip to the shelf (a network request to the server for a new HTML page).

An SPA is like having one thick binder with all the sections already in it, separated by labeled tabs. Switching from "Search" to "Settings" means flipping to a different tab — instant, no trip to the shelf needed. All the content is already in your hands.

Components are the individual sheets within each section. The "Search" tab contains a query form sheet, a results list sheet, and a pagination sheet. Each sheet handles its own content but fits into the overall binder structure.

useState is like a sticky note on the binder's cover that says "Currently viewing: Search." When you flip to a different tab, you update the sticky note, and the binder automatically opens to the right section.

The centralized API module is like having one assistant who handles all communication with the back office. Every section of the binder can say "Get me the latest config" or "Submit this search," and the assistant handles the credential badge (JWT), the delivery, and reporting back. No section needs to know where the back office is or what badge to show.

JWT-gated routing is like the binder being locked in a cabinet. Before you can flip any tabs, you have to unlock the cabinet with your badge (login). Once unlocked (token in localStorage), you have full access to all tabs. If your badge expires (token becomes invalid), the cabinet locks again and you see the login form.

Why we used it here

The Admin UI (R25) is an internal tool for configuring the search system. It needs to:

1. **Feel responsive.** Admins switch between search testing, provider settings, prompt editing, and ETL monitoring frequently. Full page reloads for each navigation would feel sluggish and would lose any unsaved state.

2. **Protect sensitive operations.** Changing AI providers or system prompts affects the entire search pipeline. JWT authentication ensures only authorized users can access these controls.
 3. **Stay simple.** This is an admin panel, not a public-facing product. URL-based routing, SEO, and browser history management are not priorities. Tab-based navigation with `useState` is simpler to build, debug, and maintain than a full routing library.
 4. **Centralize API communication.** With multiple pages making authenticated API calls, duplicating auth header logic in every component would be error-prone. The `api.js` module provides a single point of control for authentication, error handling, and base URL configuration.
-

Code walkthrough

1. The root component: JWT-gated routing (`admin-ui/src/App.jsx`)

The simplest possible auth gate:

```
import { useState } from "react";
import { isLoggedIn } from "../api.js";
import LoginPage from "../pages/LoginPage.jsx";
import Dashboard from "../pages/Dashboard.jsx";

export default function App() {
  // Check localStorage for a JWT token on mount.
  // isLoggedIn() returns true if a token exists – it does not
  // verify the token (the server does that on each API call).
  const [loggedIn, setLoggedIn] = useState(isLoggedIn());

  // If not logged in, show the login form.
  // The LoginPage receives a callback – when login succeeds,
  // it calls onLogin(), which flips loggedIn to true,
  // causing React to re-render and show the Dashboard instead.
  if (!loggedIn) {
    return <LoginPage onLogin={() => setLoggedIn(true)} />;
  }

  // Logged in – show the full admin dashboard
  return <Dashboard />;
}
```

This pattern is called **conditional rendering**. React evaluates the `if` statement on every render: if `loggedIn` is false, return the login page; otherwise return the dashboard. There is no router, no URL matching — just a boolean.

2. Tab-based navigation (`admin-ui/src/pages/Dashboard.jsx`)

The Dashboard manages which content panel is visible using tab state:

```
// Navigation structure defined as data, not hardcoded in JSX.
// This makes it easy to add/remove/reorder sections.
const NAV_SECTIONS = [
  {
    label: "Main",
    items: [
      { id: "search", label: "Search Test", icon: "\u{1F50D}" },
      { id: "analytics", label: "Analytics", icon: "\u{1F4CA}" },
      { id: "etl", label: "ETL Pipeline", icon: "\u{2699}" },
    ],
  },
  {
    label: "Configuration",
    items: [
      { id: "providers", label: "AI Providers", icon: "\u{1F916}" },
      { id: "prompts", label: "System Prompts", icon: "\u{1F4DD}" },
      { id: "users", label: "User Management", icon: "\u{1F465}" },
    ],
  },
  // ...
];

export default function Dashboard() {
  // "tab" holds the id of the currently active section.
  // Default is "search" – the first thing an admin sees.
  const [tab, setTab] = useState("search");

  return (
    <div className="app-layout">
      {/* Sidebar: renders nav buttons from the data structure */}
      <aside className="sidebar">
        <nav className="sidebar-nav">
          {NAV_SECTIONS.map((section) => (
            <div key={section.label}>
              <div className="sidebar-section-label">{section.label}</div>
              {section.items.map((item) => (
                <button
                  key={item.id}
                  // "active" class highlights the current tab
```

```

        className={`sidebar-link ${
          tab === item.id ? "active" : ""
        }}`
      // Clicking a nav button updates the tab state
      onClick={() => setTab(item.id)}
    >
      <span className="nav-icon">{item.icon}</span>
      <span>{item.label}</span>
    </button>
  )))
</div>
)))
</nav>

{/* Sign out button */}
<div className="sidebar-footer">
  <button className="sidebar-link" onClick={logout}>
    Sign out
  </button>
</div>
</aside>

{/* Main content: conditionally render the active component */}
<main className="main-content">
  <HealthStatus />
  {tab === "search" && <SearchTest />}
  {tab === "providers" && <ProviderSettings />}
  {tab === "prompts" && <PromptEditor />}
  {tab === "etl" && <EtlProgress />}
  {tab === "analytics" && <Analytics />}
  {tab === "users" && <UserManagement />}
  {tab === "guide" && <TestingGuide />}
</main>
</div>
);
}

```

The pattern `{tab === "search" && <SearchTest />}` is a JSX idiom: if `tab` equals `"search"`, the right side is evaluated and rendered; otherwise the expression short-circuits to `false`, and React renders nothing. Only one tab component is mounted at a time.

3. Centralized API module with token injection (`admin-ui/src/api.js`)

All HTTP communication funnels through this module:

```

// Token management – localStorage persists across page reloads
function getToken() {
  return localStorage.getItem("token");
}

function setToken(token) {
  localStorage.setItem("token", token);
}

function clearToken() {
  localStorage.removeItem("token");
}

function isLoggedIn() {
  return !!getToken(); // Convert to boolean: truthy if token exists
}

// The core fetch wrapper – every authenticated call goes through here
async function apiFetch(path, options = {}) {
  const token = getToken();
  const headers = {
    "Content-Type": "application/json",
    ...options.headers,
  };
  // Inject the Bearer token if we have one.
  // This is the key benefit of centralization: no component
  // needs to know about auth headers.
  if (token) {
    headers["Authorization"] = `Bearer ${token}`;
  }

  const resp = await fetch(`${API_BASE}${path}`, { ...options, headers });

  // Global 401 handler: if the server says "unauthorized"
  // (expired JWT, invalid token, etc.), clear the token
  // and reload – this drops the user back to the login page.
  if (resp.status === 401) {
    clearToken();
    window.location.reload();
    throw new Error("Session expired");
  }

  return resp;
}

```

Individual API functions are clean and focused:

```

// Login does NOT use apiFetch – it is the one call that
// happens before we have a token
async function login(username, password) {
  const resp = await fetch(`${API_BASE}/auth/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password }),
  });
  if (!resp.ok) {
    const body = await resp.json().catch(() => ({}));
    throw new Error(body.detail || "Login failed");
  }
  const data = await resp.json();
  setToken(data.access_token); // Store the JWT for future requests
  return data;
}

// Config operations use apiFetch – token is injected automatically
async function listConfig() {
  const resp = await apiFetch("/admin/config/");
  return resp.json();
}

async function setConfig(key, value) {
  const resp = await apiFetch(`/admin/config/${key}`, {
    method: "PUT",
    body: JSON.stringify({ value }),
  });
  if (!resp.ok) {
    const body = await resp.json().catch(() => ({}));
    throw new Error(body.detail || "Failed to save config");
  }
  return resp.json();
}

// Search endpoints do NOT need auth (SPEC.md §3)
// so they use raw fetch, not apiFetch
async function searchCategories(query, language, limit = 10, ...) {
  const resp = await fetch(`${API_BASE}/search/category`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ query, language: language || undefined, limit }),
  });
  return resp.json();
}

```

Notice the deliberate split: `apiFetch` is used for admin endpoints that require JWT auth, while direct `fetch` is used for search endpoints that are public (per SPEC.md section 3). This matches the backend's architecture where search routes have no auth middleware.

4. Logout (`admin-ui/src/api.js`)

```
function logout() {  
  clearToken();           // Remove JWT from localStorage  
  window.location.reload(); // Triggers App re-render  
  // App calls isLoggedIn() -> returns false -> shows LoginPage  
}
```

This is the SPA equivalent of "log out and redirect to login." Since `App.jsx` checks `isLoggedIn()` on every mount, reloading the page after clearing the token drops the user back to the login screen.

How to explain this in an interview

"The Admin UI is a React SPA with JWT-gated routing — the root App component checks localStorage for a token and conditionally renders either the Login page or the Dashboard. The Dashboard uses tab-based navigation with `useState` rather than a URL router, since this is an internal admin tool where bookmarkable URLs are not a priority. All API communication goes through a centralized module that injects the JWT Bearer token into every authenticated request and handles 401 responses globally by clearing the token and forcing a reload back to the login screen. The navigation structure is data-driven — tab sections and items are defined as a plain JavaScript array that the sidebar renders with `map`, making it easy to add or reorder sections without touching the rendering logic."